22

BÖLÜM: VERITABANI MIMARISI, BÖLÜMLEME VE REPLIKASYON | MODULE: DATABASE ARCHITECTURE, SHARDING & REPLICATION TOPOLOGIES

Veritabanı Bölümleme (Sharding) ve Yatay Parçalama: Range, Hash ve Directory

Database Sharding & Horizontal Partitioning: Range, Hash & Directory

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram

```
[ 1. RANGE-BASED SHARDING (Aralık Tabanlı) ]
Shard 1: Kullanıcı A - H  |  Shard 2: Kullanıcı I - P  |  Shard 3: Kullanıcı Q - Z
(Sıralı sorgular hızlıdır; ancak 'J' harfi aşırı popülerse Shard 2 patlar - Hotspot!)

[ 2. HASH-BASED SHARDING (Karma Tabanlı) ]
MurmurHash3(User_ID) % 4
Key: "usr_101" ⟶ Hash: 3948192 ⟶ % 4 = Shard 0
Key: "usr_102" ⟶ Hash: 8192841 ⟶ % 4 = Shard 1
(Veriyi homojen dağıtır; ancak aralık sorguları (Range Query) imkansızlaşır!)
```

💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Devasa bir kütüphanedeki 10 milyon kitabı tek bir odaya sığdıramadığınızda ne yaparsınız? Kütüphaneyi 3 odaya bölersiniz (Sharding): 1) Range Sharding (Harfe Göre): 1. odaya A-H arası yazarlar, 2. odaya I-P arası konur. Düzenlidir ama o sene 'Harry Potter' patlarsa H odasının kapısında izdiham çıkar (Hotspot), diğer odalar bomboş kalır! 2) Hash Sharding (Rastgele Matematiksel Dağıtım): Kitabın adını matematiksel bir mikserden (Hash fonksiyonu) geçirir ve odalara eşit dağıtırsınız. Odaların yükü dengelenir ama 'Bana 1990'lardaki tüm kitapları getir' derseniz 3 odayı da tek tek aramak zorunda kalırsınız!

🇬🇧 ENGLISH (ELI5)

When 10 million books overflow a library, you split them across 3 separate wings (Sharding): 1) Range Sharding: Wing 1 takes authors A-H, Wing 2 takes I-P, Wing 3 takes Q-Z. It keeps ranges alphabetical, but if J.K. Rowling blows up, Wing 2 is crushed under mob traffic (Hotspotting) while Wing 3 is dead empty! 2) Hash Sharding: You run the book ISBN through a mathematical blender (Hash function) and scatter copies evenly across all 3 wings. Traffic distributes uniformly, but executing a range query ('Find all books written between 1990 and 1995') forces you to search every single room!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Sharding (Yatay Bölümleme), tek bir veritabanının kaldıramayacağı büyüklükteki verileri ve QPS'i bağımsız veritabanı düğümlerine (Shard) bölmektir. Yöntemler: 1) Range-Based Sharding: ID veya tarihe göre aralıklarla bölme (ör. 1-100K Shard 1, 100K-200K Shard 2). Aralık sorguları (Range Scan) verimlidir; ancak sıralı ID'lerde en son shard sürekli sıcak kalır (Hotspot). 2) Hash-Based (Key-Based) Sharding: `Hash(ShardKey) % ShardSayısı`. Veriyi homojen dağıtır, hotspot'ları engeller; ancak shard sayısı değiştiğinde yeniden veri taşıma (Resharding) felaketi yaşanır (Bkz: Consistent Hashing). 3) Directory-Based: Merkezi bir arama tablosunda hangi anahtarın hangi shard'da olduğu tutulur; esnektir ama tablo tek hata noktası (SPOF) olabilir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Sharding partitions massive datasets across autonomous database instances. Strategies: 1) Range-Based Sharding: Allocates discrete contiguous intervals (e.g. A-F, G-M, or date brackets). Excellent for range scans, but susceptible to non-uniform data skew and write hotspots on append-only primary keys. 2) Hash-Based Sharding: Computes `hash(shard_key) % num_shards`. Guarantees uniform distribution and squashes hotspots, but ruins range queries (requires scatter-gather across all shards) and triggers massive rebalancing overhead when expanding nodes. 3) Directory-Based Sharding: Uses a centralized lookup service mapping entity IDs to target shards; highly adaptable but introduces an operational lookup hop and potential bottleneck.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Hash Tabanlı Sharding Yönlendirici (Shard Router)
public class ShardRouter {
    private final int numberOfShards;

    public ShardRouter(int numberOfShards) {
        this.numberOfShards = numberOfShards;
    }

    public int resolveShardId(String shardKey) {
        // MurmurHash benzeri dağıtık pozitif hash simülasyonu
        int hash = shardKey.hashCode() & 0x7fffffff;
        return hash % numberOfShards;
    }
}

void main() {
    var router = new ShardRouter(4); // 4 Veritabanı Parçası (Shard 0, 1, 2, 3)
    IO.println("user_1001 -> Shard #" + router.resolveShardId("user_1001"));
    IO.println("user_1002 -> Shard #" + router.resolveShardId("user_1002"));
    IO.println("user_1003 -> Shard #" + router.resolveShardId("user_1003"));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Sharding'in 3 büyük bedeli vardır: 1) Cross-shard JOIN imkansızlaşır (uygulama katmanında yapılmalıdır). 2) Dağıtık işlemler (Distributed Transactions / 2PC) korkunç yavaştır. 3) Shard Rebalancing (yeni makine ekleyip veriyi taşımak) operasyonel bir kabustur. Bu bedelleri mülakatta açıkça sayın!

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Articulate the 3 heavy penalties of sharding: 1) Cross-shard joins become computationally impossible. 2) Cross-shard transactions require agonizingly slow Two-Phase Commits (2PC). 3) Resharding data across new nodes is an operational minefield.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Sharding, tek bir makinenin disk ve IO limitleri aşıldığında zorunlu son çaredir.

• Hash-based sharding veriyi eşit dağıtır ancak aralık sorgularını (range query) zorlaştırır.

• Shard Key seçimi (kullanıcı ID vs şirket ID) mimarinin başarısını belirleyen en kritik karardır.

🎯 Key Takeaways for the Interview

• Sharding is the ultimate architectural frontier when single-node IO limits are exhausted.

• Hash sharding prevents write skew at the cost of requiring scatter-gather range queries.

• Selecting the Shard Key determines query locality and system longevity.